

RELATÓRIO DO PROJETO MYFOOD – MILESTONE 01

Disciplina: Programação 2

Projeto: MyFood – Milestone 01

Linguagem: Java

Aluno: Artur Ferreira Marques da Silva

Professor: Mario Hozano Lucas de Souza

Período: 2026.1

1. Introdução

Este relatório apresenta a implementação do Milestone 01 do projeto MyFood, desenvolvido na disciplina de Programação 2, utilizando a linguagem Java. O objetivo desta etapa do projeto foi construir a base de um sistema de gerenciamento de pedidos, usuários, empresas e produtos, respeitando as histórias de usuário definidas para o primeiro marco da atividade.

O sistema foi implementado de forma compatível com os testes de aceitação disponibilizados para o projeto, contemplando as funcionalidades cobradas até a US 4_2, conforme o escopo do Milestone 01. Além disso, durante o desenvolvimento, buscou-se aplicar os principais conceitos estudados em sala de aula, especialmente os fundamentos de programação orientada a objetos, como encapsulamento, herança, abstração, composição, uso de listas, tratamento de exceções e organização em pacotes.

Este documento descreve a arquitetura geral do sistema, seus principais componentes, a interação entre eles, os conceitos de orientação a objetos aplicados e o padrão de projeto utilizado.

2. Descrição geral do design arquitetural do sistema

O projeto foi organizado de forma modular, separando as responsabilidades em diferentes pacotes e classes. A arquitetura escolhida teve como objetivo principal facilitar a compreensão do sistema, evitar concentração excessiva de responsabilidades em uma única classe e tornar o código mais legível e manutenível.

A estrutura principal do sistema foi dividida em:

- **models:** contém as entidades centrais do domínio do problema;
- **services:** contém classes responsáveis pela lógica de negócio e pela persistência;
- **exceptions:** contém a exceção personalizada utilizada no projeto;
- **Facade:** classe de acesso principal utilizada pelos testes de aceitação;
- **Main:** classe responsável por executar os testes via EasyAccept.

Essa organização permite uma separação clara entre:

- os dados do sistema;
- as regras de negócio;
- o tratamento de erros;
- a persistência em arquivo;
- e o ponto de entrada externo do sistema.

Em vez de concentrar toda a lógica em uma única classe, o sistema foi dividido em serviços especializados, cada um responsável por uma parte do domínio. Essa decisão torna o design mais coerente com os princípios de orientação a objetos estudados em sala, especialmente a ideia de delegar responsabilidades às classes adequadas.

3. Principais componentes e suas interações

3.1 Classe Usuario

A classe Usuario representa o conceito geral de usuário do sistema. Ela foi implementada como classe abstrata, pois não faz sentido instanciar um usuário genérico. O sistema trabalha, de forma concreta, com dois tipos específicos de usuário: cliente e dono de empresa.

Essa classe centraliza os atributos comuns entre os usuários:

- id
- nome
- email
- senha
- endereco

Também define o método abstrato `ehDono()`, que obriga as subclasses a informarem se o usuário é ou não dono de empresa.

3.2 Classes Cliente e DonoEmpresa

As classes Cliente e DonoEmpresa herdam de Usuario.

A classe Cliente representa o usuário que realiza pedidos. Já a classe DonoEmpresa representa o usuário responsável pelo cadastro e administração de empresas no sistema. Essa segunda classe possui, além dos atributos herdados, o atributo específico `cpf`.

A principal vantagem dessa estrutura é o reaproveitamento de atributos e comportamentos comuns, evitando duplicação de código e fortalecendo o uso de herança no projeto.

3.3 Classe Empresa

A classe Empresa representa os estabelecimentos cadastrados no sistema. Ela armazena:

- id
- nome
- endereco
- tipoCozinha
- idDono
- lista de identificadores de produtos associados à empresa

Essa classe participa de uma relação de composição com produtos, pois uma empresa possui vários produtos cadastrados. Para isso, foi utilizada uma lista de ids de produtos.

3.4 Classe Produto

A classe Produto representa os itens que podem ser cadastrados em uma empresa e posteriormente adicionados a pedidos. Seus principais atributos são:

- id
- nome
- valor
- categoria
- idEmpresa

Além dos getters, a classe possui setters para permitir atualização de dados do produto, como nome, valor e categoria.

3.5 Classe Pedido

A classe Pedido representa os pedidos realizados pelos clientes. Ela armazena:

- numero
- idCliente
- idEmpresa
- estado
- lista de ids de produtos

O estado do pedido é iniciado como "aberto" e pode posteriormente ser alterado para "preparando" quando o pedido é fechado.

Assim como no caso da empresa, o pedido utiliza composição com produtos por meio de lista.

3.6 Classe DadosSistema

A classe DadosSistema funciona como um agregador central dos dados do sistema. Nela estão armazenados:

- mapa de usuários;

- mapa de empresas;
- mapa de produtos;
- mapa de pedidos;
- contadores dos próximos ids e números gerados.

Essa classe é importante porque concentra o estado geral da aplicação e facilita a persistência em arquivo, já que todo o sistema pode ser salvo e recuperado a partir desse único objeto.

3.7 Classe PersistenciaService

A classe PersistenciaService é responsável por salvar, carregar e apagar os dados persistidos do sistema. A persistência foi feita em arquivo local, garantindo que o sistema mantenha seu estado entre execuções e atenda aos testes de persistência exigidos no projeto.

Essa classe isola a lógica de entrada e saída de arquivo, evitando que esse comportamento fique espalhado pelo restante do código.

3.8 Classes de serviço

O sistema foi dividido em quatro serviços principais:

- UsuarioService
- EmpresaService
- ProdutoService
- PedidoService

Cada um desses serviços contém as regras de negócio referentes à sua área.

UsuarioService

Responsável por:

- criação de clientes;
- criação de donos de empresa;
- validação de dados;
- login;
- consulta de atributos de usuário.

EmpresaService

Responsável por:

- criação de empresas;
- listagem de empresas por dono;
- consulta de atributos de empresa;
- busca de empresa por nome e índice.

ProdutoService

Responsável por:

- criação de produtos;
- edição de produtos;
- consulta de atributos de produto;
- listagem de produtos por empresa.

PedidoService

Responsável por:

- criação de pedidos;
- adição de produtos em pedidos;
- consulta de atributos de pedidos;
- fechamento de pedidos;
- remoção de produtos;
- busca de pedidos por cliente e empresa.

3.9 Classe MyFoodException

A classe MyFoodException é uma exceção personalizada criada para representar erros de regra de negócio do sistema, como:

- email duplicado;
- nome inválido;
- produto inexistente;
- empresa não cadastrada;
- tentativa de criar pedido inválido;
- entre outros.

Seu uso melhora a clareza do código, pois permite representar erros do próprio domínio da aplicação de maneira mais organizada.

3.10 Classe Facade

A classe Facade é o ponto de entrada principal do sistema. Ela é a classe utilizada pelos testes de aceitação e funciona como uma interface simplificada para o restante da aplicação.

Seu papel não é concentrar toda a lógica, mas sim:

- inicializar o sistema;
- delegar chamadas para os serviços corretos;
- oferecer os métodos públicos esperados pelos testes;
- controlar operações gerais, como zerar e encerrar o sistema.

3.11 Classe Main

A classe Main é responsável por executar os arquivos de teste utilizando o EasyAccept. Ela não contém regra de negócio do sistema, servindo apenas como ponto de execução para validação das funcionalidades implementadas.

4. Interações entre os componentes

O funcionamento do sistema segue um fluxo bem definido.

Os testes de aceitação executam métodos da classe Facade. A Facade, por sua vez, encaminha cada operação para o serviço correspondente. Os serviços manipulam os objetos armazenados em DadosSistema. Quando necessário, o sistema usa PersistenciaService para salvar ou carregar os dados em arquivo.

De forma resumida, o fluxo principal é:

Testes → Facade → Services → DadosSistema / Entidades → Persistência

Essa organização reduz o acoplamento entre os testes e as regras internas do sistema, além de facilitar a manutenção e o entendimento do projeto.

5. Conceitos de Programação Orientada a Objetos aplicados

5.1 Encapsulamento

Os atributos das classes foram definidos como private, sendo acessados por métodos públicos apropriados. Isso impede acesso indevido ao estado interno dos objetos e contribui para manter a consistência dos dados.

5.2 Herança

Foi aplicada herança entre Usuario, Cliente e DonoEmpresa. Isso permitiu reutilizar a estrutura comum dos usuários e especializar apenas o que fosse específico de cada tipo.

5.3 Classe abstrata

A classe Usuario foi implementada como abstrata, pois representa uma ideia geral de usuário, mas não um tipo concreto a ser instanciado diretamente. Essa decisão também permitiu trabalhar melhor o conceito de abstração ensinado em sala.

5.4 Polimorfismo

O método abstrato ehDono() foi sobrescrito em Cliente e DonoEmpresa, permitindo comportamento diferente conforme o tipo concreto do objeto.

5.5 Composição

As classes Empresa e Pedido mantêm listas relacionadas a outros elementos do sistema. Isso representa relações do tipo “tem-vários”, trabalhando composição entre objetos.

5.6 Uso de listas

Foram utilizadas listas para armazenar coleções de ids, especialmente em empresas e pedidos. Isso está alinhado com o conteúdo visto sobre listas em Java.

5.7 Tratamento de exceções

O sistema utiliza exceções para sinalizar erros de validação e de regras de negócio. Isso evita retornos inconsistentes e torna a lógica de erro mais organizada e explícita.

6. Padrão de projeto adotado

6.1 Nome do padrão: Facade

6.2 Descrição geral

O padrão Facade fornece uma interface simplificada para um conjunto mais complexo de classes e subsistemas. Em vez de o código externo interagir diretamente com diversas classes internas, ele passa a interagir com uma única classe centralizadora.

6.3 Problema resolvido

No contexto do projeto, os testes de aceitação precisam acessar o sistema por meio de uma única classe, sem conhecer sua estrutura interna. Se os testes precisassem interagir diretamente com usuários, empresas, produtos, pedidos e serviços, o sistema se tornaria mais acoplado e difícil de usar externamente.

6.4 Identificação da oportunidade

A oportunidade de aplicar esse padrão surgiu pela própria exigência do projeto, que demanda uma classe Facade compatível com o EasyAccept. Assim, havia a necessidade de uma interface única e simplificada para expor as operações do sistema.

6.5 Aplicação no projeto

A classe Facade foi implementada como ponto de acesso central. Ela expõe os métodos públicos esperados pelos testes, como criação de usuários, login, criação de empresa, criação de produto, criação de pedido e assim por diante. Internamente, ela delega a responsabilidade para os serviços apropriados, mantendo a interface externa simples e o design interno mais organizado.

7. Persistência de dados

A persistência do sistema foi implementada em arquivo local, garantindo que os dados permaneçam salvos entre diferentes execuções do programa. Essa funcionalidade foi necessária para atender às histórias de usuário relacionadas à persistência no Milestone 01.

A classe `PersistenciaService` foi responsável por:

- carregar os dados previamente salvos;
- salvar o estado atual do sistema;
- apagar os dados quando o sistema é reiniciado.

Essa abordagem foi suficiente para o escopo do projeto e compatível com os testes de aceitação fornecidos.

8. Testes executados

Foram executados com sucesso os seguintes testes de aceitação do Milestone 01:

- us1_1
- us1_2
- us2_1
- us2_2
- us3_1
- us3_2
- us4_1
- us4_2

Todos os testes foram concluídos com sucesso, comprovando que o sistema implementado atende aos requisitos esperados para esta etapa do projeto.

9. Conclusão

O projeto MyFood – Milestone 01 foi implementado com sucesso em Java, respeitando o escopo previsto para a disciplina de Programação 2. A solução desenvolvida atendeu aos testes de aceitação exigidos até a US 4_2 e aplicou conceitos fundamentais de programação orientada a objetos estudados em sala, como encapsulamento, herança, abstração, composição, listas, exceções e organização modular do código.

Além do atendimento funcional, buscou-se estruturar o sistema de forma clara, separando responsabilidades entre entidades, serviços, persistência e fachada principal. A adoção do padrão Facade foi especialmente importante para viabilizar a integração com os testes e simplificar o acesso externo ao sistema.

Dessa forma, conclui-se que o projeto atingiu os objetivos propostos para o Milestone 01, tanto do ponto de vista técnico quanto do ponto de vista de organização e aplicação dos conteúdos estudados na disciplina.